

这两个问题非常切中强化学习 (RL) 在量化实战中的痛点。你之所以会有这些疑问，是因为你潜意识里还在用***“监督学习 (分类/预测)”的思维来理解“强化学习 (控制/执行)”***。

在量化交易系统中，“预测模型”和“执行模型”在特征需求和网络结构上有着本质的区别。以下为你深度拆解背后的硬核逻辑：

问题 1：特征只有 4 个是否太少？为何不用 LSTM 提取时序特征？

核心回答：在“择时卖出”这个特定任务中，特征不是越多越好，而是“信息密度”越高越好。用精心设计的 4 个动态特征配合简单的 MLP，不仅能学好，而且收敛速度和实战效果大概率碾压 20 个特征的 LSTM。

原因 1：分类模型与 PPO 模型的“社会分工”不同

- **分类模型 (你的 V1.0 LSTM)** 是“战略参谋”。它需要纵观过去 30 天的 MACD、RSI、均线等 14 个宏观特征，来决定“这只股票明天会不会大涨”。
- **PPO 模型** 是“前线狙击手”。当参谋把股票交到它手上时，说明这只票的大趋势已经没问题了。狙击手不需要知道这只票 10 天前的 RSI 是多少，他只需要盯着瞄准镜 (当前盘口)：“我现在赚了多少钱？”、“有没有高位回落？”、“还有多久收盘？”。
- **如果你把 14 个宏观特征再喂给 PPO**：这不仅是信息冗余，更会引发强化学习中最致命的***“维度灾难 (Curse of Dimensionality)”***。PPO 试错的成本极高，特征越多，状态空间越庞大，模型越容易迷失，最终什么都学不到。

原因 2：为何弃用 LSTM？(用特征工程代替 RNN)

在你之前的仓库代码中，你大量使用了 RNN/LSTM，这在强化学习中其实是一个大坑。

- **RL + LSTM 的噩梦**：在强化学习中使用 LSTM (如 RecurrentPPO) 不仅训练速度极慢 (难以并行)，而且极易出现梯度消失和方差过大。
- **马尔可夫性 (Markov Property) 的妙用**：LSTM 的作用是“记住过去”。但在我给你的代码中，我通过**特征工程**把过去的信息直接压缩到了当前状态里。
 - 比如特征 `drawdown_from_peak` (从最高点回撤的幅度)，这个单一数值就已经完美总结了过去一段时间的价格轨迹和动能衰竭情况。
 - 当当前状态已经包含了足够的决策信息时，系统就满足了马尔可夫性。此时，一个前馈神经网络 (MLP) 就能瞬间做出最优决策，完全不需要沉重的 LSTM 结构。

问题 2：为何用“单环境 + 切片替换”，而不是多环境并行？

核心澄清：我提供的这套架构，正是为了实现最高效的“多环境并行 (Vectorized Environments)”而设计的！你可能误解了“单环境类”和“并行训练”的关系。

在你之前的理解中，可能是“有 10000 只股票，我就要实例化 10000 个 `gym.Env`”。这是非常可

怕的内存灾难。

1. 传统多环境的痛点（为每只股票建一个 Env）

如果你真的创建了 1 万个独立的环境：

- **内存爆炸**：你的内存会瞬间被撑爆。
- **无法打包**：强化学习框架（如 stable-baselines3）通常要把多个环境打包成向量环境（VecEnv）去送入 GPU 计算。把 1 万个不同长度、不同数据的环境强行打包，工程实现极其困难。

2. 新架构的降维打击：“通用环境（Generic Env）”+“剧本池”

我给你的 PPOSellTraderEnv 并不是只能单线程运行。它是一个**通用环境模板**。

它的逻辑是：

1. 把 1 万只股票的数据切片放进一个大池子（List）。
2. 你可以利用 SubprocVecEnv 瞬间启动 **16 个甚至 32 个并行的环境进程**。
3. 这 16 个环境用的都是同一个 PPOSellTraderEnv 类。
4. 当环境 1 结束了一次交易触发 reset() 时，它会去大池子里**随机抽一张新的图（新股票）**接着跑；与此同时，环境 2 也在随机抽另一只股票跑。

3. 这种设计的巨大好处：

- **极致的并行效率**：只要你的 CPU 有多少核，你就可以开多少个环境并行收集数据（将代码中的 DummyVecEnv 替换为 SubprocVecEnv 即可）。
- **解决过拟合（打破数据关联性）**：如果一个环境只负责一只股票，模型会死记硬背这只股票的走势。现在，每个环境在每次 reset 时都会随机遇到完全不同的行情。这就像让一个司机每天在随机的城市里开车，他学到的才是真正的“驾驶技术（泛化交易策略）”，而不是“背地图”。
- **工程极简**：你不需要管理 1 万个环境实例，只需管理 1 个类和 1 个数据池。

总结建议

如果你还是希望尝试 RNN/LSTM 或者加入更多特征，这是完全可以的。

但我强烈建议你遵循“先跑通最简版本（MVP），再加复杂性”的工程原则：

1. **先信我一次**：就用这 4 个特征和 MLP 跑一版。你会惊讶地发现，仅仅依靠 pnl 和 drawdown_from_peak，PPO 就能非常聪明地学会“止损”和“移动止盈”。
2. **验证并行**：在 train_trader.py 中，你可以立刻将 DummyVecEnv 换成并行形式体验一下：

Python

```
from stable_baselines3.common.vec_env import SubprocVecEnv
```

```
# 启动 8 个并行环境！每个环境都会去 trajectories 随机抽数据
```

```
env = SubprocVecEnv([lambda: PPOSellTraderEnv(trajectories=trajectories) for _ in range(8)])
```

3. **后续升级**：等最简版本在你的 2025 年 8-9 月数据上跑通，并且能在 Tensorboard 看到明显的 Reward 上升曲线后，你再去逐步把大盘特征（如大盘分时涨跌）加到那 4 个特征里变成 5 个、6 个。这才是最稳妥的量化研发路径。